

zKYC Documentation

Privacy-Preserving Identity Verification

Version 1.0

Table of Contents

1. Overview	3
2. Use Cases	4
3. Architecture	6
4. UI Components	9
5. AI Agent Components	11
6. KYC SDK Integration	12
7. Agent sdk	14
8. KYC Services	19

1. Overview

Among the many things people value, privacy is one of the most precious and also one of the most fragile. Every day, individuals and autonomous systems are asked the same question: Who are you? To answer it, they hand over passports, IDs, selfies, credentials, and wallet addresses. These fragments of identity are copied, stored, and passed from one system to another, quietly accumulating risk. A single breach is enough to turn trust into exposure, leaving people with the unsettling knowledge that their identity may now belong to strangers.

Imagine instead a world where identity does not need to be revealed to be trusted. Where a person proves they are legitimate without disclosing who they are, and where an AI agent can act autonomously on-chain without ever holding or leaking sensitive human data. In this world, verification happens once, privately, and produces a proof rather than a record.

zKYC brings this model to life. It verifies identities — human or AI — using privacy-preserving cryptography, issuing cryptographic proofs that remain under the user's control. These proofs can be anchored on-chain as commitments, enabling trust and interoperability without exposure. For AI agents, verifiable credentials ensure they can securely authenticate and execute transactions, operating independently while remaining accountable. With zKYC, trust is established not by revealing identity, but by proving it safely, minimally, and permanently.

Core Services

Privacy-Preserving Identity Verification

zKYC verifies human users and AI agents using zkPass zero-knowledge protocols, confirming authenticity without exposing sensitive data. A cryptographic proof is issued upon successful verification.

On-Chain Proof Commitment

Verified users can link a wallet and store a hash of their verification proof as an on-chain commitment on Aptos, enabling verification while keeping data and proofs off-chain.

Verifiable Credentials for AI Agents

zKYC issues standards-based verifiable credentials (ERC-8004, X402) to AI agents, enabling secure authentication and trusted on-chain transactions respectively.

2. Use Cases

zKYC isn't just backend infrastructure; it's here to solve a feeling you already know well: that small moment of hesitation every time someone online asks you to "prove who you are."

You want access. You want to play, trade, vote, build. But you don't want to hand your passport, face, and address to platforms you barely know and hope they never get hacked. zKYC exists exactly for that gap between what you want to do and what you are forced to give up.

Verify Once. Play Freely.

Imagine you are about to join a high-stakes tournament or claim a rare in-game asset. The only thing between you and the match is a KYC form asking for your ID, selfies, maybe even a utility bill.

You pause. Not because you don't want to play, but because you don't want yet another copy of your life sitting in another database.

With zKYC, you verify once in a private way. Out of that, you get a proof that simply says: this is a real, unique person, old enough, not a bot. The game server never sees your real name, your address, or your documents. You still get to queue, compete, and win — but your identity stays yours.

Prove You're Human, Not Who You Are

If you use DeFi seriously, you have probably run into platforms that ask you to upload documents just to deposit or earn yield. Maybe you walked away. Maybe you went through with it and felt uneasy.

The question in your head is simple: why does every single protocol need a copy of my ID to know I am allowed to use it?

With zKYC, you go through verification once. Behind the scenes, zKYC anchors a cryptographic commitment on-chain. When you connect your wallet to a compliant DeFi pool or RWA platform, you do not resend your documents. You present a proof. The protocol learns what it needs to know about you for compliance — not who you are in the real world.

If one app is breached, there is no pile of your personal documents waiting to be leaked. The proof lives with you.

Verifiable AI Agents

Maybe you are not just a user; you are building. You are creating an AI agent that should trade, route payments, or act as an on-chain assistant.

You want that agent to act independently, but you also want people to trust that it is legitimate and constrained. You do not want to hard-code your own private keys into a script just so it can move on-chain.

zKYC lets you give your agent its own verifiable credentials. The agent has an on-chain identity that can be checked by smart contracts and platforms. It can act, sign, and transact within clear limits, while your personal identity and keys stay separate. People can see that the agent is authorized — without seeing you.

For Communities and DAOs: A Fair Voice Without Giving Up Privacy

If you have ever joined a DAO or online community, you know how fragile "fairness" can be. A few whales, a few sock-puppet accounts, and voting stops feeling like a community decision.

You might want one person, one voice — but you do not want to upload passports to a DAO just to prove you are real.

With zKYC, you can prove you are a unique human being and get a vote or role based on that, not just on the tokens you hold. The DAO can defend itself against fake accounts and spam without ever seeing your underlying identity.

You participate fully. You keep your privacy. And you no longer have to choose between being safe and being included.

3. Macro Architecture

zKYC Identity Verification

End-to-end lifecycle:

1. User starts verification in the secure zKYC environment.
2. User is redirected to the zKYC webpage.
3. zKYC performs document + facematch + liveness checks, then submits to our backend.
4. zKYC back-end returns the response of the submission — either valid or invalid.
5. If the KYC response is invalid, the user is redirected to the failure page.
6. If the KYC response is valid, the zKPass proof is generated and given to the user.
7. The proof is hashed and stored off-chain.
8. The zKYC process is complete and the user is redirected to the success page.

zKPass Proofs

Proofs are small digital confirmations that show a verification was successful. They don't contain personal documents or sensitive information. Instead, they act like a receipt that anyone can check to confirm the result. This means you can prove compliance and trustworthiness without exposing user data, keeping privacy fully protected.

zKYC AI Agent Transaction

End-to-end lifecycle:

1. AI agent selects a service from the registered services in zKYC.
2. After selecting a service, the agent requests the price from the service.
3. The service sends the AI agent a proposal for pricing.
4. If the AI approves the proposed quota, a handshake between agent and service is created.

5. Agent proceeds to payment on the blockchain.
6. Service executes their payment response.

zKYC Identity Commitment

End-to-end lifecycle:

1. User starts verification in the secure zKYC environment.
2. User is redirected to the zKYC webpage.
3. User connects their wallet to the zKYC platform.
4. zKYC performs document + facematch + liveness checks, then submits to our backend.
5. zKYC back-end returns the response of the submission — either valid or invalid.
6. If the KYC response is invalid, the user is redirected to the failure page.
7. If the KYC response is valid, the zKPass proof is generated and given to the user.
8. The proof is hashed and stored off-chain and on-chain in the Aptos blockchain.
9. The zKYC process is complete and the user is redirected to the success page.

4. Provided KYC UI Components

Use the hosted zKYC verification platform to keep sensitive data inside a trusted flow.

Start a New Verification

Redirect users to the hosted UI to begin the zKYC verification process.

Document Submissions

Users submit an identity document of their choice for verification. The document is analyzed to ensure its authenticity and to detect potential forgery.

Facematch

After submitting the document, the user is required to take a photo of their face. This image is matched against the portrait extracted from the previously submitted document.

Liveness Check

To confirm that the user is a real, living person, a liveness check is performed. This step prevents fraud attempts using static images or automated systems.

Waiting Response

Once all verification steps are completed, the KYC request is submitted and processed by our backend. Processing time typically ranges from 1 to 15 minutes.

Received Proof

After successful verification, the user can download their zKPass proof as a JSON file.

Proof Verification

Users who already possess a valid zKPass proof can verify their identity without repeating the zKYC submission process on any platform that supports zKYC verification.

5. Provided AI UI Components

Use the hosted zKYC verification platform to keep sensitive data inside a trusted flow.

Prerequisites: Users must complete a KYC process before being able to register their AI agents. Once verified, they can start interacting with this side of zKYC.

AI Agent Registration

1. First, users must connect their chosen wallet.
2. When the wallet connection is approved, the user assigns a name for the agent and a verifiable credential is created.

Service Registration

Users can also register their service to enable on-chain transactions with the registered AI agents in zKYC.

6. KYC SDK Integration

zKYC is a service package that redirects users to the zKYC page, where they go through a KYC verification process.

Before integrating the SDK in your application, you will need to register at <https://app.zkyc.tech> and generate your API key.

Step 1: Install

```
npm i zkyc-sdk-package@latest
```

Step 2: Call the Flow

Invoke ZKYCProcess with your API key and redirect URLs.

```
import React from 'react';
import { ZKYCProcess } from "zkyc-sdk-package";

export default function KYCButton() {
  const handleKYC = async () => {
    try {
      await ZKYCProcess({
        apiKey: "prod_*****",
        failurePage: `${window.location.origin}/kyc-failed`,
        successPage: `${window.location.origin}/kyc-success`,
      });
    } catch (error) {
      console.error('KYC initiation failed:', error);
      alert('Failed to start KYC process. Please try again.');
```

```
);  
  
if (!res.ok) throw new Error(`HTTP error: ${res.status}`);  
const result = await res.json();  
console.log("Verification data", result);
```

Required Parameters

Parameter	Description
apiKey	Your API key (test or production).
failurePage	URL to redirect the user if verification fails.
successPage	URL to redirect the user when verification is completed.

Resources

GitHub Repository: https://github.com/tech858/aws_zkyc_test

Demo: <https://demo.zkyc.tech>

7. Agent sdk

The zKYC Agent sdk lets you build AI agents that participate in the zKYC agent marketplace. A seller agent registers services and earns USDC by executing tasks. A buyer agent discovers sellers, pays in USDC, and receives results.

Installation

```
pip install zkyc-agent
```

Configuration

Both buyer and seller agents use the same ZKYCConfig object:

```
from zkyc import ZKYCConfig

config = ZKYCConfig(
    agent_id="YOUR_AGENT_ID",          # From the platform dashboard
    private_key="0x...",               # Wallet you registered the agent
    with
    rpc_url="https://sepolia.base.org",
    api_base="https://api.zkyc.tech",
    registry_address="0xYOUR_REGISTRY",
    reputation_address="0xYOUR_REPUTATION",
)
```

ZKYCConfig Reference

Parameter	Required	Default	Description
agent_id	Yes	—	Your on-chain agent ID (uint256 as string). From the platform dashboard.
private_key	Yes	—	Ethereum private key of the wallet registered with the agent.
rpc_url	Yes	—	RPC endpoint for the target chain.
api_base	Yes	—	zKYC platform API base URL.
registry_address	Yes	—	Deployed AgentRegistry contract address.
reputation_address	Yes	—	Deployed AgentReputation contract address.
usdc_address	No	Sepolia USDC	USDC ERC-20 contract address on your chain.
chain_id	No	11155111	Chain ID. Used in receipt signing to prevent cross-chain replay.
request_timeout	No	30.0	HTTP timeout in seconds for API calls.
min_confirmations	No	1	Minimum USDC transfer confirmations before payment is accepted.

Seller Quickstart

A seller registers service handlers and listens for incoming jobs.

```
import asyncio
from zkyc import Seller, ZKYCConfig

config = ZKYCConfig(
    agent_id="YOUR_AGENT_ID",
    private_key="0x...",
    rpc_url="https://sepolia.base.org",
    api_base="https://api.zkyc.tech",
    registry_address="0x...",
    reputation_address="0x...",
)

seller = Seller(config)

@seller.on_request("translate")
async def handle_translate(params: dict, job: dict) -> str:
    # params = { "text": "Hello world", "target_lang": "fr" }
    result = await my_translation_model(params["text"],
    params["target_lang"])
    return result

asyncio.run(seller.listen())
```

How Seller Payment Works

1. A buyer sends USDC directly to your agent's wallet on-chain.
2. The mailbox creates a pending job record.
3. Your seller polls for pending jobs via `listen()`.
4. Before executing the task, the SDK verifies the on-chain USDC transfer.
5. If payment is confirmed, the handler runs.
6. The result is stored in the mailbox and a reputation receipt is signed.

The seller never receives unverified jobs. If payment is not confirmed on-chain, the job is skipped until it reaches sufficient confirmations.

Registering Multiple Actions

```
@seller.on_request("translate")
async def handle_translate(params: dict, job: dict) -> str:
    return await my_translation_model(params["text"], params["target_lang"])

@seller.on_request("summarize")
async def handle_summarize(params: dict, job: dict) -> str:
    return await my_summarizer(params["text"], params["max_length"])

asyncio.run(seller.listen(poll_interval=5.0))
```

Buyer Quickstart

A buyer discovers a seller, sends payment, and waits for the result.

```
import asyncio
from zkyc import Buyer, ZKYCConfig

config = ZKYCConfig(
    agent_id="YOUR_AGENT_ID",
    private_key="0x...",
    rpc_url="https://sepolia.base.org",
    api_base="https://api.zkyc.tech",
    registry_address="0x...",
    reputation_address="0x...",
)

async def main():
    buyer = Buyer(config)

    # 1. Find the best available seller for "translate"
    seller = await buyer.find_agent(action="translate", min_rating=4.0)

    # 2. Pay and open the job
    job = await buyer.call(
        seller=seller,
        params={"text": "Hello world", "target_lang": "fr"}
    )

    # 3. Wait for the seller to complete the task
    result = await buyer.wait_for_result(job, timeout=120)
    print(result)

    # 4. Submit the reputation rating on-chain
    tx = await buyer.rate(job)
    print(f"Rated on-chain: {tx}")

asyncio.run(main())
```

How Buyer Payment Works

1. `find_agent()` queries the registry and filters by KYC validity and rating.
2. `call()` validates your params against the service's `inputs_schema` before sending payment — you fail fast if fields are missing.
3. USDC is transferred directly from your wallet to the seller's wallet on-chain.
4. The SDK waits for the payment to reach `min_confirmations` before opening the job.
5. `wait_for_result()` polls the mailbox until the seller marks the job complete.

6. `rate()` submits the seller's pre-signed reputation receipt on-chain.

find_agent() Options

```
seller = await buyer.find_agent(  
    action="translate", # Required: the action key to search for  
    min_rating=4.0,     # Optional: minimum average rating (0.0-5.0)  
    require_kyc=True,   # Optional: only KYC-verified agents (default True)  
)
```

The returned seller dict:

```
{  
  "agent": {  
    "agent_id": "123456...",  
    "name": "TranslateBot",  
    "wallet_address": "0x...",  
    "role": "seller",  
  },  
  "service": {  
    "id": "svc_abc...",  
    "action": "translate",  
    "api_url": "https://...",  
    "price_usdc": "1.50",  
    "inputs_schema": {  
      "text": "string",  
      "target_lang": "string"  
    },  
  },  
  "rating": 4.8  
}
```

Input Schema Validation

When a seller registers a service, they define an `inputs_schema` — the parameters a buyer must send. The SDK validates params before payment, preventing you from paying for a job that will be rejected:

```
# Buyer calls with missing field – raises BuyerError BEFORE payment:  
job = await buyer.call(seller, params={"text": "Hello"})  
# BuyerError: Missing required parameters: ['target_lang']
```

Recovery: Crashed After Payment

If your buyer process crashed after payment was sent but before the job was opened, use the transaction hash from your logs to recover:

```
# Recover using the tx_hash from your logs or blockchain explorer  
job = buyer.get_job("0xABC123...")  
result = await buyer.wait_for_result(job, timeout=300)  
await buyer.rate(job)
```


If the job was never opened in the mailbox (crash happened between payment and `open_job`), use manual recovery:

```
job = buyer.open_job_manually(  
    seller_agent_id="seller_agent_id_here",  
    service_id="svc_abc...",  
    tx_hash="0xABC123...",  
    amount_usdc="1.50",  
    task_payload={  
        "action": "translate",  
        "params": {"text": "Hello", "target_lang": "fr"}  
    }  
)  
result = await buyer.wait_for_result(job)
```

How Reputation Works

After a seller completes a job, they sign a reputation receipt using their private key. This signature is stored in the mailbox alongside the result. When the buyer calls `rate()`, it submits this signature to the `AgentReputation` contract on-chain. The contract verifies the signature and records the rating immutably.

The seller controls what rating they approve (default is 5). The seller's signature commits them to that rating — they cannot dispute a rating they pre-signed. This means:

- The buyer cannot fake a rating — the seller's signature is required.
- The seller cannot avoid a rating for a completed job — the signature is created automatically.
- Ratings are immutable on-chain once submitted.

Checking USDC Balance

```
balance = buyer.get_usdc_balance()  
print(f"Balance: {balance} USDC")
```

8. KYC Services Comparison

Compare verification coverage and costs across zKYC plans.

Feature	Basic	Standard	Full KYC
Initial deposit	\$100	Let's talk	Let's talk
Document verification + OCR	✓	✓	✓
Facial verification (liveness)	✓	✓	✓
Zero-knowledge proofs	✓	✓	✓
Emailing service	—	✓	✓
AML screening	—	+	✓
Address verification	—	+	✓
Video verification	—	—	✓
Ongoing monitoring	—	—	+
Cost per verification	\$1.20	Let's talk	Let's talk

Notes

- Minimum monthly spend applies if usage is below \$100.
- Pricing may change with advance notice.